

Phase 0 Discovery Document

Ready Game Code Migration: Laravel to Next.js + NestJS

A production-grade migration blueprint preserving 100% feature parity with the existing Unity source-code marketplace, designed for 10,000+ concurrent users on AWS.

Document Version: Part A (v1.0)

Prepared By: Principal Software Architect

Prepared For: Ready Game Code Engineering Team

Date: July 2026

Source: Production SQL dump + live site audit

Status: Awaiting sign-off



Table of Contents

Executive Summary

1. Strategic Decisions Locked
 2. Database Analysis & Schema Decision Record
 3. Public Surface Feature Audit
 4. Architecture Recommendations
 5. AWS Deployment Topology
 6. Strangler-Fig Migration Roadmap
 7. Auth Migration Plan
 8. Data Migration Script Plan
 9. URL/SEO Preservation Plan
 10. Security Checklist
 11. Risk Register & Open Questions
 12. Next Steps & Sign-off
- Appendix A: Complete 67-Table Inventory**
- Appendix B: Active Payment Gateway Configuration**
- Appendix C: Glossary**

This document is Part A of Phase 0 Discovery. Part B (controller logic, admin flows, validation rules, email templates) will follow upon receipt of the Laravel source code.

Executive Summary

Project Context

Ready Game Code is a production Unity source-code marketplace currently running on Laravel (PHP) with MySQL. The platform facilitates the sale of Unity game source codes, services, and add-ons (reskin, publish, store optimization) between sellers (authors) and buyers, with admin-managed product approval workflows. The business now wants to migrate the entire application to a modern TypeScript stack (Next.js + NestJS) without changing any user-facing behavior, business rules, or data semantics.

Locked Strategic Decisions

Ten strategic decisions were confirmed before Phase 0 began: drop speculative modules (Wallet, Leaderboard, generic Transaction); adopt password rehash on first login for auth migration; use Prisma as ORM; deploy on AWS; preserve the existing Razorpay + PayPal + manual UPI payment mix during cutover; use strangler-fig migration; target 10,000+ concurrent users; defer search and observability tooling selection to Phase 0 recommendation; defer test coverage standard to Phase 0 recommendation. All ten decisions are documented in Section 1.

Phase 0 Status

Phase 0 is delivered in two parts. Part A (this document) is produced from the SQL database dump and a live audit of the production site at <https://readygamecode.com>. It covers database analysis, public-surface feature inventory, target architecture, AWS topology, migration roadmap, and risk register. Part B (to follow) will cover controller business logic, validation rules, middleware, admin flows, email templates, and queued jobs, and depends on the user sharing the Laravel source code, composer.json, .env.example, and admin Blade templates.

Key Findings

The database contains 67 tables organized across nine domains. Three payment gateways are active in production (Razorpay for INR, PayPal for USD, and a manual Google Pay UPI flow with QR code). The deposits table already contains an `is_web` flag, indicating that the Laravel backend is already API-enabled for a Next.js frontend in development. The reviewer role is separate from the admin role, suggesting a multi-actor approval workflow. KYC, 2FA, referral, and multi-language systems exist in the schema but are disabled in production. File storage is local disk today, with configuration hooks for FTP, Wasabi, DigitalOcean Spaces, and Backblaze.

Top Three Risks

First, sensitive payment gateway keys (Razorpay live key, PayPal client secret) are present in plaintext in the database dump and must be rotated immediately. Second, the existing Laravel business logic and admin flows cannot be fully reverse-engineered from the SQL schema alone; Part B analysis requires the source code. Third, SEO rankings must be preserved during cutover, requiring a strict URL-preservation and 301-redirect plan executed in the final phase.

Recommended Next Action

Approve Part A, rotate exposed payment keys, and share the Laravel source code so that Phase 0 Part B can be produced. Once Part B is approved, Phase 1 (Foundation) can kick off with monorepo scaffolding, design system, and the auth module.

Top finding: The deposits table contains an `is_web` field that indicates the existing Laravel backend is already API-enabled for a Next.js frontend. Existing API contracts can be reused, significantly de-risking the migration. The user should share any existing Next.js code and the Laravel routes/api.php file.

SECTION 1

Strategic Decisions Locked

All ten strategic questions raised in the migration prompt have been answered and locked. These decisions form the foundation of every subsequent section in this document and the implementation roadmap. Any change to these decisions after sign-off requires a formal Migration Decision Record (MDR) approval.

Locked Decisions Matrix

#	Decision	Choice	Rationale
Q1	Module list	18 modules (speculative dropped)	Dropped Wallet, Leaderboard, generic Transaction. Marketplace does not have wallets or leaderboards; transactions fold into Order module.
Q2	Auth migration	Password rehash on first login	Verify Laravel bcrypt \$2y\$ hash in NestJS using bcrypt lib. On success, re-hash with \$2b\$ and issue JWT. Zero session invalidation, minimal complexity.
Q3	ORM	Prisma	Cleaner schema DSL, better type inference, mature migrations, fewer decorator bugs than TypeORM. Strong fit for NestJS + MySQL in 2026.
Q4	Deployment	AWS	ECS Fargate for NestJS, RDS MySQL Multi-AZ, ElastiCache Redis, S3 + CloudFront, SES, SQS. Scales to 10k+ concurrent users with auto-scaling.
Q5	Payment flow	Razorpay (INR) + PayPal (USD) + manual UPI	Preserves existing gateway mix. Recommends adding Stripe Checkout post-cutover to replace PayPal direct (PayPal's IPN is deprecated). Manual UPI remains for India.
Q6	Cutover strategy	Strangler-fig (gradual)	Run Laravel + NestJS in parallel. Migrate page-by-page over 30 weeks. Per-route rollback. Preserves SEO, allows incremental validation.
Q7	Search backend	Meilisearch (Phase 0 recommendation)	MySQL LIKE will not scale to 10k+ concurrent users. Meilisearch is open-source, sub-50ms search, typo tolerance, easy self-host on EC2. Alternative: Typesense.
Q8	Performance target	10,000+ concurrent users	Architecture sized for 10k concurrent users, 2,000 RPS, p95 < 250ms. RDS read replicas + Redis cache + CDN for static assets + Meilisearch offload.
Q9	Observability	Sentry + CloudWatch + UptimeRobot (Phase 0 rec.)	Sentry for errors and performance traces. CloudWatch for logs/metrics/alerts (AWS-native). UptimeRobot for external uptime. Cost-effective, covers all three signals.
Q10	Test coverage	80% lines / 70% branches (Phase 0 rec.)	Minimum 80% line coverage and 70% branch coverage enforced in CI. E2E tests (Playwright) mandatory for critical paths: cart, checkout, admin approval, auth, registration.

SECTION 2

Database Analysis & Schema Decision Record

The existing MySQL database contains 67 tables organized across nine functional domains. This section provides: (a) a domain-grouped table inventory, (b) a Schema Migration Decision Record for the highest-priority tables, and (c) ten critical findings that materially affect the migration strategy.

2.1 Table Inventory by Domain

Domain	Tables	Count	Notes
Auth & Users	users, admins, reviewers, admin_password_resets, reviewer_password_resets, password_resets, user_logins, device_tokens, user_follows, author_level_user	10	Three distinct auth surfaces: users (buyers/sellers), admins (super), reviewers (product approval). Laravel bcrypt \$2y\$ hashes.
Catalog	products, categories, sub_categories, product_collections, collection_product, product_views, product_user, product_rating, changelogs, extensions, forms	11	Products are the core entity. Categories drive fees (personal vs commercial buyer fee). Sub-categories include Multiplayer, Racing, Match Tile, Puzzle, Platformer, Casual, Cards, Action.
Commerce	carts, orders, order_items, coupons, campaigns, campaign_products, deposits, withdrawals, withdraw_methods, transactions, refund_requests, refund_activities	12	Full e-commerce: cart with license picker + addon services, orders with payment_status, deposits for payment tracking, withdrawals for seller payouts, transactions as ledger.
Reviews & Social	reviews, review_categories, comments, followers, reported_reviews, reported_reviews_attachments, ratings	7	Reviews are categorized (review_categories table). Comments are nested (parent_id). Followers enable social graph. Reported reviews have moderation workflow.
Content	blog_posts, blog_categories, pages, frontends, advertisements, subscribers	6	Blog with SEO (seo_content JSON). Pages are static. Frontends is UI customization. Subscribers is newsletter list.
Admin & Workflow	activities, rejections, author_levels	3	Activities is product approval workflow (status: 0=pending, 1=approved, 5=permanent down). Rejections: hard vs soft. Author levels: gamification with minimum_earning + fee.
Payments	gateways, gateway_currencies	2	19 gateways configured in DB. Only 3 active: Razorpay (INR), PayPal (USD), Google Pay (manual). Each gateway stores credentials in gateway_parameters JSON.
Notifications	notification_logs, notification_templates, admin_notifications	3	Templates support email + SMS + push. Logs track every notification sent. Admin notifications separate from user notifications.
Support & Moderation	support_tickets, support_messages, support_attachments	3	Full ticketing system with priority levels, attachments, and admin reply workflow.

Domain	Tables	Count	Notes
Settings & System	general_settings, languages, migrations, api_keys, cron_jobs, cron_job_logs, cron_schedules, update_logs	8	general_settings is massive config table (file_server, mail_config, sms_config, socialite_credentials, referral, KYC toggles, etc.). Cron system with logs.

2.2 Schema Migration Decision Record (Priority Tables)

Decision values: **KEEP** = preserve table as-is in new schema. **KEEP + ADD COLUMN** = preserve and add new fields. **REFACTOR** = restructure (e.g., split into multiple tables, introduce parent entity). **RENAME** = same structure, new name for clarity. **EVALUATE** = pending source code review. **DEPRECATE** = drop in new schema.

Table	Decision	Prisma Entity	Notes
users	KEEP + ADD COLUMN	User	Add: auth_provider, auth_provider_id, password_algo (default 'bcrypt'), last_login_at. Keep bcrypt \$2y\$ hashes; rehash on first NestJS login.
admins	KEEP	Admin	Map to AdminRole. Use separate JWT guard. Add: last_login_at.
reviewers	KEEP	Reviewer	Separate role from Admin. subcategories field is JSON array. Add: last_login_at.
products	KEEP + ADD COLUMN	Product	Add: search_index_updated_at, prisma_search_vector. Convert tags text to separate ProductTag table for efficient filtering.
categories	KEEP	Category	seo_content JSON kept. fees (personal/commercial/extended) kept as Decimal.
sub_categories	KEEP	SubCategory	form_id links to dynamic forms (subcategory_attributes). Keep as-is.
carts	REFACTOR	CartItem	Cart is per-item, not per-cart. Consider introducing Cart parent entity with CartItem children for cleaner checkout logic. reskin_selected, publish_selected, store_optimization_selected flags kept.
orders	KEEP + ADD COLUMN	Order	Add: gateway, gateway_transaction_id, currency_code, ip_address, user_agent. payment_status: 0=pending, 1=paid.
order_items	KEEP	OrderItem	purchase_code is unique download code. license: 1=personal, 2=commercial. seller_earning is computed at order time.
transactions	KEEP	Transaction	Ledger table. trx_type '+' = credit, '-' = debit. remark: payment/purchase/new_sale/balance_add. Indexed by user_id + trx.
deposits	REFACTOR	Payment	Rename to Payment for clarity. is_web field already supports Next.js. status: 1=success, 2=pending, 3=cancel. Stores gateway-specific JSON in detail.

Table	Decision	Prisma Entity	Notes
gateways	KEEP	Gateway	19 gateways configured. Only 3 active (status=1). Credentials in gateway_parameters JSON. CRITICAL: rotate Razorpay live key + PayPal secret immediately.
reviews	KEEP	Review	review_category_id links to review_categories. is_reported flag for moderation.
comments	KEEP	Comment	Nested comments via parent_id. author_reply flag indicates seller response.
blog_posts	KEEP	BlogPost	seo_content is JSON with CHECK constraint (json_valid). body is longtext. is_published flag.
activities	KEEP	Activity	Product approval workflow log. status: 0=pending, 1=approved, 5=permanent down. message contains action type prefix [Approve]/[Update Approved]/[Permanent Down].
rejections	KEEP	Rejection	type: 2=hard reject, 3=soft reject. Linked to reviewer_id.
refund_requests	KEEP	RefundRequest	Linked to order_item_id. status workflow with refund_activities log.
support_tickets	KEEP	SupportTicket	status: 0=Open, 1=Answered, 2=Replied, 3=Closed. priority: 1=Low, 2=Medium, 3=High.
general_settings	KEEP (single row)	SiteSetting	Singleton config table. Contains 50+ settings. Split into typed Prisma model with JSON fields for complex configs (mail_config, sms_config, socialite_credentials).
notifications_logs	RENAME	NotificationLog	Truncate to NotificationLog for naming consistency. user_read flag.
followers	KEEP	Follow	Social graph: user_id follows follower_id. Counter cached on users.total_follower/total_following.
author_levels	KEEP	AuthorLevel	Gamification: minimum_earning threshold + fee. Linked via author_level_user pivot.
coupons	KEEP	Coupon	discount_type: 1=fixed, 2=percent. usage_limit_per_coupon and usage_limit_per_user.
forms	KEEP	DynamicForm	Custom form builder. form_data is JSON. Used for author_info, subcategory_attributes, withdraw_method.
pages	KEEP	Page	Static pages: name, slug, body content. Used for About, Privacy, Terms, Refund Policy.
product_views	EVALUATE	(maybe deprecate)	Counter cache vs analytics. If only used for total_views counter, replace with Redis hyperloglog. Confirm with source code review.
product_user	KEEP	UserPurchase	Tracks which user has purchased/downloaded which product. Used for download access control.
downloads	KEEP	DownloadLog	Download tracking. Linked to product_user. Add: ip_address, user_agent for security audit.
subscribers	KEEP	Subscriber	Newsletter subscribers. Add: unsubscribed_at, source (footer/popup/checkout).

2.3 Critical Database Findings

Ten findings materially affect migration strategy, security posture, or feature scope. Each finding includes the impact and recommended action.

Finding 1: Laravel is already API-enabled for Next.js

The deposits table contains an ``is_web`` field with the comment 'This will be 1 if the request is from NextJs application'. This means the Laravel backend already serves API endpoints for a Next.js frontend (likely in development or partially deployed). Implication: existing API contracts can be discovered and replicated, reducing migration risk. The user should share any existing Next.js code and the Laravel routes/api.php file.

Finding 2: Multiple payment gateways, only 3 active

The gateways table has 19 rows (PayPal, Stripe, Skrill, PayTM, Payeer, PayStack, Flutterwave, Razorpay, Stripe Storefront, Instamojo, Blockchain, CoinPayments, Coingate, Coinbase, BTCPay, NowPayments, Binance, SslCommerz, Aamarpay, bKash, MercadoPago, Authorize.net, NMI, 2Checkout, Checkout, Mollie, Cashmaal, Google Pay, PayPal SDK, Stripe V3). Only 3 are status=1 (active): Razorpay (code 110, INR), PayPal (code 101, USD), and Google Pay (code 1000, manual UPI). Recommendation: keep Razorpay + PayPal during migration, plan Stripe Checkout as a post-cutover replacement for PayPal direct (PayPal IPN is being deprecated).

Finding 3: Sensitive credentials exposed in database dump

The gateways table contains live production credentials in plaintext within the gateway_parameters JSON field. Confirmed exposed: Razorpay live key_id (rzp_live_SEgBGBF6PO3ZLf) and key_secret, Stripe test secret_key and publishable_key, PayPal client_id and client_secret, Google OAuth client_id and client_secret (from general_settings.socialite_credentials). Action required: rotate all exposed credentials before any code is written. This is a P0 security incident.

Finding 4: Three-role auth system

Three separate tables store authenticated users: users (buyers/sellers), admins (super-admin), and reviewers (product approval role). Each has its own password field with bcrypt \$2y\$ hashes. The reviewers table has a subcategories JSON field assigning which sub-categories each reviewer can approve (e.g., reviewer Pranav is assigned to subcategories [1,2,3,4,5,6,7]). The auth migration plan must preserve all three roles and map them to NestJS guards.

Finding 5: Author levels gamification

The author_levels table defines seller tiers based on minimum_earning threshold, with each tier having a different fee percentage. The author_level_user pivot table assigns users to tiers. Current state: only one author level exists in production. This is a forward-looking gamification system that should be preserved but may need business rules confirmed via source code.

Finding 6: Disabled features (KYC, 2FA, Referral, Multi-language)

The `general_settings` table contains toggles for KYC verification (`kv=0`), 2FA (`ts` field, currently off), referral system (`referral=0`), and multi-language (`multi_language=1` but only English is configured). The `users` table has corresponding fields (`kyc_data`, `kv` status, `ts` 2FA toggle, `tv` 2FA verified). These features are dormant in production but the schema supports them. Recommendation: preserve schema, do not implement feature logic unless explicitly requested.

Finding 7: File storage is local disk, configurable

`general_settings.file_server=1` (current = local disk). The schema supports FTP (`file_server=2`), Wasabi (`file_server=3`), DigitalOcean Spaces (`file_server=4`), and Backblaze (`backblaze` field). Migration target: AWS S3 with CloudFront CDN. The APK files and product thumbnails should be migrated to S3 during Phase 3 (Commerce) before cutover. Total file size inventory required.

Finding 8: Email uses PHP mail(), configurable

`general_settings.mail_config` is JSON `{"name":"php"}`, indicating PHP `mail()` function. The schema supports SMTP and other drivers. Migration target: AWS SES with NestJS nodemailer. Email templates are stored in `notification_templates` table (`email_body` field) and `general_settings.email_template` (global wrapper). Templates use `{{fullname}}`, `{{username}}`, `{{message}}` shortcodes.

Finding 9: Cron jobs are DB-driven

The `cron_jobs` table stores scheduled tasks with `cron_schedule_id` reference, `next_run` timestamp, `is_running` flag, and `is_default` flag. The `cron_schedules` table defines intervals. The `cron_job_logs` table records execution history with duration and error. Migration target: AWS EventBridge + Lambda for serverless cron, OR BullMQ repeatable jobs on the NestJS worker process. Source code review needed to enumerate exact cron jobs and their logic.

Finding 10: Dynamic form builder

The `forms` table stores dynamic form definitions as JSON. Three forms exist: `withdraw_method` (empty), `author_info` (5 questions: team size, members, marketplace accounts, categories), and `subcategory_attributes` (empty). These forms are rendered dynamically on the frontend. Migration: build a `DynamicForm` React component that parses the `form_data` JSON and renders appropriate inputs. Form submissions should be stored as JSON in a new `form_submissions` table.

SECTION 3

Public Surface Feature Audit

Every public-facing URL was audited via a headless browser session against the live production site (<https://readygamecode.com>). This section catalogs: (a) the complete URL map with auth requirements, (b) page-by-page feature breakdown, and (c) cross-cutting features present on every page. Admin-internal pages were not directly auditable without source code access and are listed as TBD.

3.1 URL Map

URL Path	Page Type	Auth Required	Notes
/	Homepage	No	Hero, search, info cards, featured products, popular items, blog teaser
/products	Catalog listing	No	Sort + filters (search, price, category, date), 9-page pagination
/products/{slug}	Product detail	No	Gallery, video, APK download, license picker, addon services, comments, related
/products?category={slug}	Filtered catalog	No	Category filter via query param
/products?sort={new best Rated best selling}	Sorted catalog	No	Sort via query param
/cart	Cart	No (session-based)	Order summary, checkout button (disabled when empty)
/checkout	Checkout	Yes	Payment gateway selection, coupon, final review
/checkout/thank-you	Order confirmation	Yes	Receipt + download links
/user/login	User login	No	Google OAuth + email/password. REDIRECT to /login in Next.js
/user/register	User registration	No	Google OAuth + email/password + terms agreement. REDIRECT to /register
/blog	Blog listing	No	Posts grouped by category, SEO slugs
/blog/{slug}	Blog post detail	No	Article body, cover image, SEO meta, related posts
/about	About page	No	Gamification-themed metrics, roadmap
/contact	Contact form	No	Form + FAQ accordion
/admin	Admin login	No (admin auth)	Separate login surface. Username/password only
/privacy-policy	Privacy Policy	No	Static page from pages table
/terms-conditions	Terms & Conditions	No	Static page
/refund-policy	Refund Policy	No	Static page
/resources	Resources hub	No	Likely blog or docs category. Confirm with source
/game	Game category page	No	Mega-menu link. Likely /products?category=game

URL Path	Page Type	Auth Required	Notes
/services	Services category page	No	Returns 404 currently — likely broken or upcoming. Investigate
/hire-us	Hire us (lead form)	No	Project inquiry form. Likely stored in support_tickets
/author/{username}	Author profile	No	Seller portfolio, sales stats, followers, ratings
/user/dashboard	User dashboard	Yes	Purchases, downloads, profile settings
/user/purchases	Purchase history	Yes	Order history with download links
/user/downloads	Download center	Yes	Purchased product files
/user/refunds	Refund requests	Yes	Buyer refund workflow
/seller/dashboard	Seller dashboard	Yes	Sales, earnings, product management
/seller/products	Seller products	Yes	CRUD for seller's own products
/seller/earnings	Seller earnings	Yes	Transaction ledger + withdrawal requests
/seller/withdrawals	Seller withdrawals	Yes	Withdrawal request + history

3.2 Page-by-Page Feature Breakdown

Page	Feature	Implementation Notes
Homepage	Hero banner with tagline	'Turn Your Game Dreams into Reality with Premium Source Codes!' — server-rendered, A/B test-ready
Homepage	Hero search bar	Searches products by title, tags, description. Targets Meilisearch index in new arch
Homepage	3 info cards	Helpful resources, Unity marketplace, Trusted support — static content blocks
Homepage	Featured products carousel	Products where is_featured=1. Paginated server-side, infinite scroll on client
Homepage	Popular items carousel	Products ordered by total_sold DESC. Cached in Redis for 5 min
Homepage	Latest blog teaser	3 most recent blog_posts where is_published=1, ordered by published_at DESC
Products listing	Sort dropdown	Options: New Item (created_at DESC), Best Rated (avg_rating DESC), Best Selling (total_sold DESC)
Products listing	Sidebar filters	Search box, Min/Max price spinbuttons, Category links, Date radio (Any/Year/Month/Week/Day)
Products listing	Pagination	20 items per page. Laravel-style pagination links (1, 2, 3... 9). Cursor-based recommended for new arch
Products listing	Product card	Thumbnail, title, vendor name, add-to-cart button, Live Preview link (opens demo_url)
Product detail	Image gallery + thumbnails	Multiple preview images per product. Lightbox on click
Product detail	Video preview (YouTube embed)	preview_video field stores YouTube URL. Lazy-loaded
Product detail	APK download link	Download APK for testing. Served from /storage/. New arch: signed S3 URLs with 1-hour expiry
Product detail	License picker (Personal vs Commercial)	Radio buttons. Updates price dynamically. Maps to products.price vs products.price_cl
Product detail	Addon services (Reskin, Publish, Store Optimization)	3 checkboxes. Prices: \$120, \$25, \$50. Maps to products.reskin_price, publish_price, store_optimization
Product detail	Add to Cart button	POST to /cart. Stores in carts table with license, addon flags. Session-based for guests
Product detail	Author box	Vendor name (Ready Game Code), portfolio link, sales count, follower button
Product detail	Comments / Reviews tab	Two tabs: Description, Comments. Reviews use reviews table; comments use comments table (nested)
Product detail	More items by author	Thumbnail strip of 8 products by same user_id
Product detail	Sidebar widgets	Why this source code is useful, What you get, Best for, Helpful next steps, Tags

Page	Feature	Implementation Notes
Cart	Order summary	Subtotal, discount, total. Checkout button disabled when cart empty
Cart	Remove items	AJAX delete. Updates cart counter in header
Checkout	Payment method selection	Razorpay (INR), PayPal (USD), Google Pay UPI (manual QR)
Checkout	Coupon code	Input + apply button. Validates against coupons table
Checkout	Final review	Order items, totals, billing email. Place Order button triggers gateway redirect
Login	Google OAuth button	Laravel Socialite. Google client_id in socialite_credentials. Reuse same OAuth app
Login	Email/password	Username OR email. Remember me checkbox. Forgot password link
Register	First name, Last name, Email, Password, Confirm	Terms agreement checkbox (Privacy + Terms + Refund Policy). Google OAuth option
Blog	Category chips	ASO & Publishing, Marketing & Growth, etc. Filter blog_posts by blog_category_id
Blog	Post cards	Cover image, category, title, excerpt, read more link. Paginated
Contact	Contact form	Name, Email, Subject, Message. Creates support_ticket with status=0
Contact	FAQ accordion	5 commonly asked questions. Static content, expandable
Admin	Admin login	Username/password only. No OAuth. Separate session from user
Admin	(TBD - source code needed)	Admin dashboard, product approvals, user management, settings, refunds, support tickets, etc.

3.3 Cross-Cutting Features

Feature	Location	Notes
Header navigation	Every page	Logo, All Items, About, Resources, Game (mega-menu), Services, Privacy Policy, Terms, Register, Login, Cart counter
Footer	Every page	Quick Links, Policy Pages, Get In Touch (email, phone, WhatsApp), Social links (Facebook, X, LinkedIn, Instagram, YouTube)
Cart counter	Header	Shows item count. AJAX-updated. Session-based for guests, user-based for logged-in
Hire us CTA	Header	Lead-gen form. Likely creates support_ticket with priority=Low
Social links	Footer	Facebook, X (Twitter), LinkedIn, Instagram, YouTube. Stored in general_settings or hard-coded
WhatsApp chat	Footer	+91 9408212310. Direct deep link to wa.me
Cookie consent	(if present)	GDPR compliance. Confirm with source code review
Maintenance mode	general_settings.maintenance_mode	Toggle to take site offline. Implement as Next.js middleware that checks Redis flag
Multi-language toggle	(disabled)	general_settings.multi_language=1 but only English configured. Implement i18next-ready structure

SECTION 4

Architecture Recommendations

The target architecture is a modular monolith NestJS backend with a Next.js App Router frontend, deployed on AWS, sized for 10,000+ concurrent users. The system uses Prisma as ORM, JWT in httpOnly cookies for auth, Meilisearch for search, and Redis for caching. The 18 NestJS modules below correspond directly to the database domains identified in Section 2.

4.1 NestJS Module Breakdown

Each module is a bounded context containing controllers, services, repositories (Prisma), DTOs, entities, guards, and module-scoped providers. Shared infrastructure (Prisma, Redis, Queue, Logger) is centralized in a SharedModule imported by all feature modules.

Module	Key Entities	Controllers	Services	Notes
Auth	User, Admin, Reviewer, RefreshToken	AuthController (login, register, refresh, logout, forgot-password, reset-password, verify-email), OAuthController (google)	AuthService, JwtStrategy, RefreshTokenService, PasswordService (bcrypt rehash), OAuthService	Three auth surfaces: user, admin, reviewer. Each with own guard. JWT in httpOnly cookies.
User	User, UserLogin, DeviceToken, UserFollow	UsersController (profile, update, dashboard), UserFollowsController (follow, unfollow)	UserService, UserFollowsService	Profile, dashboard, follower graph. Counter cache fields (total_follower, total_following).
Product	Product, ProductView, ProductTag, ProductCollection, Changelog	ProductsController (list, detail, create, update, delete), ProductUploadsController (file upload), ProductCollectionsController	ProductsService, ProductSearchService (Meilisearch), ProductViewService, ProductUploadService (S3)	Core module. 50+ fields per product. Search via Meilisearch. File storage on S3. Slug-based URLs.
Category	Category, SubCategory	CategoriesController, SubCategoriesController	CategoriesService	Drives fee structure (personal_buyer_fee, commercial_buyer_fee, twelve_month_extended_fee).
Cart	Cart, CartItem (new)	CartsController (add, update, remove, list, clear)	CartsService, CartPricingService (calculates license + addon totals)	Refactor: introduce Cart parent entity. Session-based for guests, user-based for logged-in. Migrate session cart to user cart on login.
Order	Order, OrderItem	OrdersController (create, list, detail, cancel)	OrdersService, OrderPricingService, OrderNumberService	Atomic transaction: create order + order_items + decrement stock + clear cart. Idempotent via idempotency key.

Module	Key Entities	Controllers	Services	Notes
Payment	Payment (renamed from Deposit), Gateway, GatewayCurrency	PaymentsController (initiate, callback, webhook), GatewaysController (admin)	PaymentsService, RazorpayService, PaypalService, ManualUpiService, PaymentWebhookService	Webhook signature verification mandatory. Idempotent webhook processing via webhook_event_id. Store gateway_payload JSON for audit.
License	License (computed), LicenseFeature	(no separate controller — embedded in Product and OrderItem)	LicenseService (calculates price based on Personal vs Commercial + addon services)	Addon prices: Reskin \$120, Publish \$25, Store Optimization \$50. Stored per-product, overridable.
Review	Review, ReviewCategory, Comment, ReportedReview, ReportedReviewAttachment, Rating	ReviewsController (CRUD, report), CommentsController (CRUD, reply), ReportedReviewsController (admin)	ReviewsService, CommentsService, ReviewModerationService	Nested comments via parent_id. Author reply flag. Reported reviews workflow.
Blog	BlogPost, BlogCategory	BlogController (list, detail, by-category), BlogCategoriesController (admin)	BlogService, BlogSeoService	SEO-critical: seo_content JSON, slug, cover_image, published_at. SSR via Next.js for indexability.
Page	Page, Frontend, Advertisement	PagesController (by-slug), FrontendsController (admin)	PagesService	Static pages (About, Privacy, Terms, Refund). Frontends is UI customization blocks.
Author	User (as Author), AuthorLevel, AuthorLevelUser, Follower	AuthorsController (profile, portfolio, stats), AuthorLevelsController (admin)	AuthorsService, AuthorEarningsService, AuthorLevelService	Seller profile. Author levels gamification. Follower social graph.
Activity	Activity, Rejection	ActivitiesController (list, by-product), RejectionsController (admin create)	ActivitiesService, ProductApprovalService	Product approval workflow. status: 0=pending, 1=approved, 5=permanent down. Reviewer role action.
Admin	Admin, AdminNotification, AdminPasswordReset	AdminController (login, dashboard, users, products, orders, payments, refunds, settings, support, blog)	AdminAuthService, AdminDashboardService, AdminUserService, AdminProductService	Separate JWT guard. All endpoints under /api/v1/admin/*. Audit log every admin action.
Settings	SiteSetting (singleton), Language, ApiKey	SettingsController (read), AdminSettingsController (update)	SettingsService (cached in Redis, 5min TTL), LanguagesService	Singleton config. 50+ fields. Split into typed Prisma model. Cache reads in Redis.
Notification	NotificationLog, NotificationTemplate	NotificationsController (list, mark-read), AdminNotificationTemplateController	NotificationsService, EmailService (SES), SmsService (Twilio), PushService (FCM), TemplateRenderer	Multi-channel: email + SMS + push. Templates use {{shortcodes}}. Queue via BullMQ.
Media	MediaAsset (new), DownloadLog	MediaController (upload, download), DownloadsController	MediaService (S3 + CloudFront), DownloadUrlService (signed URLs, 1hr expiry), ApkDownloadService	S3 + CloudFront for assets. Signed URLs for paid downloads. Track download log for audit.

Module	Key Entities	Controllers	Services	Notes
Support	SupportTicket, SupportMessage, SupportAttachment	SupportTicketsController (create, list, detail, reply, close), AdminSupportTicketsController	SupportTicketsService, SupportAttachmentService (S3 upload)	Priority: Low/Medium/High. Status: Open/Answered/Replied/Closed. Attachments on S3.
Shared/Common	(no entities — utilities only)	(no controllers — global interceptors/filters/pipes)	ConfigService, LoggerService, RedisService, PrismaService, QueueService, HashService, PaginationService, ResponseService	Cross-cutting. Global exception filter, validation pipe, logging interceptor, rate-limiter guard, correlation-id middleware.

4.2 Frontend Structure (Next.js App Router)

Next.js uses feature-based folder structure under /app. Server Components by default, Client Components only where interactivity is required ('use client'). React Query (TanStack Query) for server state, Axios for API calls, Zod for runtime validation. OpenAPI client auto-generated from NestJS Swagger spec for end-to-end type safety.

```

app/
  (public)/                # public route group
    page.tsx               # homepage
    products/
      page.tsx             # product listing
      [slug]/page.tsx      # product detail
    blog/
      page.tsx
      [slug]/page.tsx
    about/page.tsx
    contact/page.tsx
    privacy-policy/page.tsx
    terms-conditions/page.tsx
    refund-policy/page.tsx
  (auth)/                  # auth route group
    login/page.tsx
    register/page.tsx
    forgot-password/page.tsx
  (user)/                  # authenticated user
    dashboard/page.tsx
    dashboard/purchases/page.tsx
    dashboard/downloads/page.tsx
    dashboard/refunds/page.tsx
    dashboard/settings/page.tsx
  (seller)/                # seller role
    seller/page.tsx
    seller/products/page.tsx
    seller/products/new/page.tsx
    seller/earnings/page.tsx
    seller/withdrawals/page.tsx
  (admin)/                 # admin role
    admin/login/page.tsx
    admin/dashboard/page.tsx
    admin/products/page.tsx
    admin/users/page.tsx
    admin/orders/page.tsx
    admin/refunds/page.tsx

```

```

    admin/settings/page.tsx
  cart/page.tsx
  checkout/page.tsx
  checkout/thank-you/page.tsx
  hire-us/page.tsx
  authors/[username]/page.tsx
components/
  ui/                # buttons, inputs, cards (shadcn/ui)
  product/           # product-specific components
  cart/              # cart components
  admin/             # admin components
  layout/            # header, footer, navigation
lib/
  api/               # generated API client
  hooks/             # React Query hooks
  utils/             # utilities
  validations/       # Zod schemas

```

4.3 API Design Principles

- RESTful endpoints under **/api/v1/*** — versioned from day 1 for future evolution.
- OpenAPI 3.1 spec auto-generated from NestJS Swagger decorators, consumed by Next.js for type-safe client code.
- Idempotency keys required on all POST `/payments`, POST `/orders`, POST `/refunds` — prevents double-charge on retry.
- Pagination: cursor-based (not offset) for performance on large tables. Default page size 20, max 100.
- Filtering: RFC 3986-style query params (`?filter[status]=paid&sort=-created_at`).
- Error format: RFC 7807 Problem Details — `{ type, title, status, detail, instance }`.
- Rate limiting: 100 req/min/user, 1000 req/min/IP. 429 response with Retry-After header.
- Response envelope: `{ data, meta: { pagination, request_id } }`. Errors use Problem Details.
- All write endpoints accept JSON only. Multipart for file uploads. No XML anywhere.
- Webhook endpoints (Razorpay, PayPal) accept raw body, verify HMAC signature, return 200 immediately and process async.

4.4 Auth Flow (Password Rehash Strategy)

On the first NestJS login attempt, the system receives the plaintext password, verifies it against the stored Laravel bcrypt `$2y$` hash, and on success immediately re-hashes with `$2b$` at cost 12 and updates the user record. The user is then issued JWT access (15min) + refresh (7d) tokens in httpOnly cookies. Subsequent logins use the modern `$2b$` hash directly. No session invalidation is required — users experience zero disruption.

4.5 Shared Module Split

The SharedModule is split into three sub-modules to maintain separation of concerns: (1) KernelModule — domain primitives, types, enums, constants. (2) InfrastructureModule — Prisma, Redis, S3, SQS, SES clients. (3) CommonModule — DTOs, decorators, interceptors, filters, guards. Each feature module imports only what it needs.

SECTION 5

AWS Deployment Topology

The AWS architecture is designed for 10,000+ concurrent users with Multi-AZ high availability, horizontal auto-scaling, and cost optimization. The total monthly cost is estimated at approximately USD 2,898, dominated by RDS MySQL (Multi-AZ + read replica). All resources are provisioned via Terraform (Infrastructure as Code) and deployed via GitHub Actions CI/CD.

5.1 Topology Components

Layer	AWS Service	Configuration	Purpose
Edge / CDN	CloudFront	Origins: S3 (static assets), ALB (dynamic). Price class 200. WAF attached.	Global content delivery, DDoS protection, SSL termination
DNS	Route 53	Hosted zone for readygamecode.com. Weighted routing during cutover.	DNS resolution, health checks, gradual traffic shifting
WAF	AWS WAF	Rules: AWS Managed Core, rate-limit 2000/IP, block SQLi/XSS, geographic restrictions optional.	L7 protection, bot mitigation, rate limiting
Frontend hosting	Vercel OR S3 + CloudFront + Lambda@Edge	Option A (recommended): Vercel Pro for Next.js (App Router SSR + Edge functions). Option B: S3 for static + Lambda@Edge for SSR.	Next.js hosting with SSR, ISR, edge middleware
Backend compute	ECS Fargate (NestJS API)	Auto-scaling 4-20 tasks. CPU 70% target. ALB in front. 2 vCPU / 4GB per task.	NestJS API server. Horizontal auto-scaling based on CPU + RPS.
Background workers	ECS Fargate (NestJS Worker)	Auto-scaling 2-8 tasks. Runs BullMQ workers for email, image processing, search indexing.	Async job processing
Database	RDS MySQL 8.0	db.r6g.2xlarge (8 vCPU, 64GB RAM). Multi-AZ. 1 read replica. Automated backups 30 days. Performance Insights enabled.	Primary datastore. Multi-AZ for HA. Read replica for read scaling.
Cache	ElastiCache Redis	cache.r6g.large. Cluster mode disabled (single shard). 1 replica. Multi-AZ.	Session store, cart cache, settings cache, rate-limit counter, Meilisearch cache
Search	Meilisearch on EC2 (or Meilisearch Cloud)	Self-hosted: c6i.2xlarge (8 vCPU, 16GB). EBS gp3 100GB. Auto-snapshot daily. OR Meilisearch Cloud (managed) — \$99/mo for 10M documents.	Full-text search across products, blog, pages. Sub-50ms queries.
Object storage	S3	3 buckets: rgc-media (public, CloudFront), rgc-apks (private, signed URLs), rgc-backups (private). Lifecycle: transition to IA after 30 days, Glacier after 90 days.	Product images, APK files, screenshots, blog covers, user uploads, backups
Queue	Amazon SQS	3 queues: emails, image-processing, search-indexing. DLQ per queue. Visibility timeout 300s.	Async task queue for BullMQ
Email	Amazon SES	Production access in us-east-1. Custom MAIL FROM domain. DMARC/DKIM/SPF configured.	Transactional emails (welcome, order confirmation, refund, password reset)

Layer	AWS Service	Configuration	Purpose
Secrets	AWS Secrets Manager + Parameter Store	Secrets Manager: DB password, JWT secret, gateway keys, OAuth secrets. Parameter Store: non-sensitive config (feature flags, API URLs).	Centralized secrets management. Auto-rotation for DB password.
Networking	VPC	10.0.0.0/16. Public subnets (ALB, NAT). Private subnets (ECS, RDS, Redis, Meilisearch). 3 AZs. VPC Flow Logs enabled.	Network isolation. Private resources not internet-accessible.
CI/CD	GitHub Actions + ECR	Pipeline: lint -> test -> build -> push to ECR -> deploy to ECS (blue-green). Manual approval for production. Automated PR previews on Vercel.	Source-to-prod automation with quality gates
Monitoring	CloudWatch + Sentry + UptimeRobot	CloudWatch: RDS/ECS/Redis metrics + alarms. Sentry: app errors + performance. UptimeRobot: external pings every 1 min from 6 regions.	Three signals: metrics, errors, uptime
Logging	CloudWatch Logs	Log groups per service. Retention 90 days. Structured JSON logs. Correlation ID in every log.	Centralized application logs

5.2 Monthly Cost Estimate

Costs are estimated for steady-state production traffic. Burst traffic (e.g., product launch, marketing campaign) may add 20-40% to compute and egress costs. Reserved Instances (1-year, no upfront) for RDS and ElastiCache can reduce those line items by 25-35%.

Service	Configuration	Monthly Cost (USD)
Vercel Pro (Next.js)	Team plan + 1TB bandwidth	\$40
ECS Fargate (API)	4 tasks x 2 vCPU/4GB always-on + scaling	\$280
ECS Fargate (Worker)	2 tasks x 1 vCPU/2GB	\$70
ALB	1 ALB + 50GB data transfer	\$40
RDS MySQL	db.r6g.2xlarge Multi-AZ + 1 replica + 200GB storage	\$1,200
ElastiCache Redis	cache.r6g.large + replica	\$180
EC2 (Meilisearch)	c6i.2xlarge + 100GB EBS	\$250
S3	500GB standard + 1TB CloudFront transfer	\$90
CloudFront	1TB egress (after S3 credit)	\$85
SQS	10M requests	\$5
SES	100K emails	\$40
Secrets Manager	20 secrets + 1M API calls	\$40

Service	Configuration	Monthly Cost (USD)
Route 53	1 hosted zone + 1M queries	\$8
WAF + Shield	WAF rules + Shield Advanced	\$300
CloudWatch	Logs + Metrics + Alarms	\$50
Sentry + UptimeRobot	Team plans	\$80
NAT Gateway	2 NAT gateways + 100GB	\$140
TOTAL (estimated)		~\$2,898/month

5.3 High Availability & DR

Multi-AZ deployment for RDS (synchronous standby) and ElastiCache (replica). ECS Fargate tasks distributed across 3 AZs. ALB routes traffic with health checks. Meilisearch on a single EC2 instance (warm standby in second AZ for manual failover). S3 is inherently 11-nines durable. RPO: 5 minutes (RDS point-in-time recovery). RTO: 30 minutes. Full DR runbook documented in a separate Disaster Recovery Plan.

SECTION 6

Strangler-Fig Migration Roadmap

Migration follows the strangler-fig pattern: the new Next.js + NestJS system grows around the existing Laravel application, replacing it page-by-page over 30 weeks. Both systems run in parallel during the transition, with Route 53 weighted routing controlling traffic split. Each phase has clear scope, deliverables, acceptance criteria, and per-phase rollback. The Laravel application remains warm for 7 days post-cutover as a fallback.

6.1 Six-Phase Roadmap

Phase	Weeks	Scope	Deliverables	Acceptance Criteria
Phase 1: Foundation	1-4	Monorepo (Turborepo), design system, auth module, user module, static pages (about, contact, privacy, terms).	Working Next.js + NestJS skeleton. Login + register + Google OAuth. Static pages live. CI/CD pipeline. Vercel + ECS deploy.	User can register, login, view profile. All static pages render. Lighthouse > 90. Tests at 80% coverage.
Phase 2: Catalog	5-10	Product module, category module, search (Meilisearch), product detail page, product listing page, category browse, author profile.	Product listing live with sort + filters. Product detail live with gallery, video, addons. Meilisearch index built. Author profile live.	Product pages load < 1s. Search returns results < 50ms. SEO parity with Laravel site. Sitemap regenerated.
Phase 3: Commerce	11-16	Cart module, order module, payment module (Razorpay + PayPal + UPI), license module, checkout flow, thank-you page, downloads.	Cart with license picker + addons. Checkout with payment gateway. Order confirmation with download links. Webhook handlers verified.	End-to-end purchase works for all 3 gateways. Webhooks verified with test transactions. Seller earnings credited.
Phase 4: Engagement	17-20	Review module, comment module, blog module, followers, notifications, support tickets.	Reviews + nested comments live. Blog listing + detail live. Follow/unfollow works. Email notifications sent.	Review submission works. Email delivery confirmed via SES. Support ticket flow complete.
Phase 5: Admin & Ops	21-26	Admin module, reviewer workflow (activities), rejections, refund workflow, admin settings, admin support, cron jobs migration.	Admin login + dashboard live. Product approval workflow tested. Refund flow tested. Cron jobs running on EventBridge.	Admin can approve/reject products. Refund flow end-to-end tested. All cron jobs executing on schedule.
Phase 6: Cutover	27-30	DNS cutover, full traffic to new stack, Laravel read-only (for fallback), monitoring, rollback rehearsal.	DNS flipped. 100% traffic on new stack. Old Laravel kept warm for 7 days as fallback. Monitoring dashboards live.	No regression in user-facing behavior. p95 latency < 250ms. Error rate < 0.1%. SEO rankings stable for 14 days.

6.2 Per-Phase Top Risk & Mitigation

Phase	Top Risk	Mitigation
Phase 1	Auth migration breaks existing user logins	Test password rehash on staging with 100% of production users. Dry-run before any cutover.
Phase 2	SEO ranking drops during catalog migration	Preserve all URLs exactly. Use 301 redirects only if URL must change. Submit sitemap immediately. Monitor GSC.
Phase 3	Payment webhook signature verification fails	Test with gateway sandbox first. Have manual verification runbook. Keep Laravel payment flow active as fallback.
Phase 4	Email delivery lands in spam	Warm up SES domain 2 weeks before. Configure DMARC/DKIM/SPF. Monitor bounce rate < 2%.
Phase 5	Admin workflow regression causes product approval backlog	Shadow-run both systems for 1 week (admins use new, Laravel still active). Compare audit logs.
Phase 6	DNS cutover causes downtime	Use Route 53 weighted routing: 5% new for 24h, 25% for 48h, 100% after. Auto-rollback if error rate > 1%.

6.3 Parallel-Run Strategy

During Phases 2-5, both Laravel and Next.js serve traffic. Route 53 weighted routing controls the split: 5% Next.js / 95% Laravel for the first 24 hours of each phase, expanding to 50/50 after 72 hours, and 100/0 after 1 week of error-free operation. Auto-rollback triggers: error rate > 1% on Next.js route, p95 latency > 500ms, or any critical user flow regression. Rollback is automatic via CloudWatch alarm -> Lambda -> Route 53 weight change.

SECTION 7

Auth Migration Plan

Current State

The Laravel application uses bcrypt with the \$2y\$ prefix (PHP's identifier) at cost factor 12. Three tables store user credentials: `users.password` (buyers/sellers), `admins.password` (super-admins), and `reviewers.password` (product approvers). All three must be migrated. The bcrypt \$2y\$ hash is cryptographically identical to the \$2b\$ hash used by Node.js's bcrypt library — they differ only in the prefix identifier. Node.js bcrypt can verify \$2y\$ hashes with a minor version-tolerant configuration.

Migration Strategy: Password Rehash on First Login

On the first NestJS login attempt for each user, the system will: (1) receive the plaintext password from the user, (2) verify it against the stored \$2y\$ hash using bcrypt's compare function, (3) on success, immediately re-hash the plaintext with \$2b\$ at cost 12, (4) update the user record with the new hash and set a `password_algo` field to 'bcrypt-2b', (5) issue JWT access + refresh tokens. On subsequent logins, the system checks `password_algo` and uses the modern verification path. No session invalidation is required; users experience zero disruption.

Role Mapping

Three roles map to three NestJS guards: (1) `UserJwtGuard` for buyers/sellers from the `users` table, (2) `AdminJwtGuard` for super-admins from the `admins` table, (3) `ReviewerJwtGuard` for product approvers from the `reviewers` table. Each guard validates a separate JWT issued by its own login endpoint. RBAC is enforced via `Casl.js` with role-based abilities. The reviewer role has additional subcategory-scoped permissions (`reviewers.subcategories` JSON field) — a reviewer can only approve products in their assigned subcategories.

JWT Strategy

Access tokens are short-lived (15 minutes) and contain { `sub`, `role`, `email` }. Refresh tokens are long-lived (7 days), stored in a `refresh_tokens` table with rotation on use, and bound to the user's device fingerprint. Both tokens are delivered in `httpOnly`, `Secure`, `SameSite=Lax` cookies. The frontend never sees the token strings directly. CSRF protection is via the `SameSite` attribute plus a double-submit CSRF token for state-changing requests.

OAuth Migration

Google OAuth is currently configured via Laravel Socialite (`client_id` 174714211-v1uh0r6tp3fn6kb9j0jucvk5tevc365s). The same OAuth app will be reused — no new Google credentials needed. NestJS will use `Passport.js` with the `google-oauth20` strategy. On OAuth callback, NestJS checks if a user with that `google_id` exists; if yes, issues JWT; if no, creates a new user account. Facebook and LinkedIn Socialite configs exist in the database but are disabled (`status=0`). They will be schema-preserved but not implemented in Phase 1.

Session Invalidation Plan

No forced session invalidation is required. The silent re-auth approach means users continue using Laravel until their respective pages are migrated to Next.js. Once a user hits a Next.js page that requires auth, they will be prompted to log in (since Laravel session cookie is not shared with Next.js). On first Next.js login, their password is rehashed. Users who never log in to Next.js retain their \$2y\$ hash indefinitely (no security risk — \$2y\$ is still cryptographically sound).

7.1 Auth Migration Sequence Diagram (Textual)

```
User -> Next.js login form: enters email + password
Next.js -> NestJS /api/v1/auth/login: POST { email, password }
NestJS AuthService:
  1. Find user by email in users table
  2. Read stored password hash (e.g., '$2y$12$xfd2U...')
  3. bcrypt.compare(plaintext, hash) -- Node bcrypt supports $2y$
  4. If match: re-hash with bcrypt.hash(plaintext, 12) -> '$2b$12$...'
  5. UPDATE users SET password = $2b_hash, password_algo = 'bcrypt-2b' WHERE id = ?
  6. Generate JWT access (15min) + refresh (7d)
  7. Set httpOnly + Secure + SameSite=Lax cookies
  8. Return { user: { id, email, name, role } }
Next.js -> User: dashboard rendered
```

SECTION 8

Data Migration Script Plan

Strategy

Data migration uses a two-phase approach: (1) Schema migration via Prisma migrations — creates all tables, indexes, and constraints in the new database. (2) Data migration via custom Node.js scripts using Prisma's batch create — reads from old MySQL, transforms if needed, writes to new MySQL. Both databases run on the same RDS instance during migration (different schemas) to minimize network transfer time.

Pre-Migration Checklist

Before any migration: (1) Take RDS automated snapshot, label as pre-migration-baseline. (2) Enable RDS binary logging for point-in-time recovery. (3) Put site in maintenance mode (`general_settings.maintenance_mode=1`) to freeze writes. (4) Notify users 72 hours in advance. (5) Run migration dry-run on staging with full production data clone. (6) Verify all scripts have idempotent `INSERT...ON DUPLICATE KEY UPDATE` semantics.

Migration Order (dependency-aware)

Tables must be migrated in dependency order to satisfy foreign key constraints: (1) `general_settings`, `languages`, `forms`, `gateways`, `gateway_currencies` (configuration), (2) `users`, `admins`, `reviewers`, `author_levels` (auth), (3) `categories`, `sub_categories` (taxonomy), (4) `products`, `product_collections`, `changelogs`, `extensions` (catalog), (5) `carts`, `coupons`, `campaigns` (commerce-setup), (6) `orders`, `order_items`, `deposits`, `transactions`, `withdrawals`, `refund_requests` (commerce-data), (7) `reviews`, `review_categories`, `comments`, `reported_reviews` (engagement), (8) `blog_posts`, `blog_categories`, `pages`, `frontends` (content), (9) `activities`, `rejections` (workflow), (10) `support_tickets`, `support_messages`, `support_attachments` (support), (11) `notification_logs`, `notification_templates` (notifications), (12) `followers`, `user_follows` (social). Each batch is wrapped in a transaction; on failure, batch is rolled back and logged.

Validation Queries

Post-migration validation runs automated reconciliation: (1) Row count comparison: `SELECT COUNT(*)` for every table, old vs new, must match exactly. (2) Sum reconciliation: `SELECT SUM(amount)` from `transactions`, `orders`, `deposits` — must match to the cent. (3) Referential integrity: every `order.user_id` exists in `users.id`; every `order_item.product_id` exists in `products.id`. (4) Hash check: MD5 hash of (`id`, `email`, `created_at`) for `users` table, old vs new. (5) Sample 100 random orders and verify all related `order_items`, `transactions`, `deposits` exist in new DB. Validation report is auto-generated and must be signed off by engineering lead before cutover.

Rollback Procedure

If validation fails or critical regression is found post-cutover: (1) Switch DNS back to Laravel (Route 53 weighted routing — instant). (2) Laravel comes back online with pre-migration data (writes during migration window are lost — acceptable since maintenance mode was on). (3) RDS point-in-time recovery available if data corruption occurred. (4) Decision to retry migration vs abort made within 4 hours of rollback. (5) Post-mortem within 48 hours. Rollback is rehearsed in staging before production cutover.

Estimated Downtime

Total production downtime: 30 minutes. Breakdown: (1) Enable maintenance mode: 1 min. (2) Final incremental data sync (delta from dry-run): 5 min. (3) Run validation queries: 5 min. (4) Switch DNS to Next.js: 2 min. (5) Smoke tests on production: 10 min. (6) Disable maintenance mode: 1 min. (7) Buffer: 6 min. If any step exceeds budget, abort and rollback.

SECTION 9

URL/SEO Preservation Plan

SEO preservation is a first-class migration concern. The current Laravel site has established search engine rankings that must be preserved through cutover. This section provides the URL mapping (old to new), the 301 redirect plan for changed URLs, and the SEO parity checklist.

9.1 URL Mapping & Redirects

Old URL	New URL	Type	Notes
/	/	EXACT MATCH	No change. Homepage preserved
/products	/products	EXACT MATCH	No change. Query params preserved
/products/{slug}	/products/{slug}	EXACT MATCH	No change. Slug is primary key
/cart	/cart	EXACT MATCH	No change
/checkout	/checkout	EXACT MATCH	No change
/checkout/thank-you	/checkout/thank-you	EXACT MATCH	No change
/blog	/blog	EXACT MATCH	No change
/blog/{slug}	/blog/{slug}	EXACT MATCH	No change
/about	/about	EXACT MATCH	No change
/contact	/contact	EXACT MATCH	No change
/privacy-policy	/privacy-policy	EXACT MATCH	No change
/terms-conditions	/terms-conditions	EXACT MATCH	Verify exact slug from pages table
/refund-policy	/refund-policy	EXACT MATCH	Verify exact slug from pages table
/user/login	/login	301 REDIRECT	Shorter URL. Laravel returns 301 to /login
/user/register	/register	301 REDIRECT	Shorter URL
/user/dashboard	/dashboard	301 REDIRECT	Shorter URL
/user/purchases	/dashboard/purchases	301 REDIRECT	Nested under dashboard
/user/downloads	/dashboard/downloads	301 REDIRECT	Nested under dashboard
/user/refunds	/dashboard/refunds	301 REDIRECT	Nested under dashboard
/seller/dashboard	/seller	301 REDIRECT	Shorter URL
/seller/products	/seller/products	EXACT MATCH	No change
/seller/earnings	/seller/earnings	EXACT MATCH	No change
/seller/withdrawals	/seller/withdrawals	EXACT MATCH	No change
/author/{username}	/authors/{username}	301 REDIRECT	Plural for consistency
/admin	/admin	EXACT MATCH	No change. Separate Next.js route group
/admin/login	/admin/login	EXACT MATCH	No change (Laravel returns /admin page)
/resources	/resources	EXACT MATCH	No change. Confirm target with source code

Old URL	New URL	Type	Notes
/game	/products?category=game	301 REDIRECT	Map mega-menu link to filtered catalog
/services	/products?category=services	301 REDIRECT	Currently 404 — investigate
/hire-us	/hire-us	EXACT MATCH	No change. Lead form

9.2 SEO Parity Checklist

Every public page in the new Next.js application must satisfy the following checklist before cutover. The checklist is enforced in CI via Lighthouse CI + Playwright SEO assertions.

- Meta title preserved per page (40-60 chars, brand suffix pattern)
- Meta description preserved per page (140-160 chars)
- OpenGraph tags (og:title, og:description, og:image, og:url) on every public page
- Twitter Card tags (twitter:card, twitter:title, twitter:description, twitter:image)
- Canonical URLs on every page (prevent duplicate content)
- Sitemap.xml auto-generated from products + blog_posts + pages, submitted to Google Search Console
- Robots.txt with disallow /admin, /api, /dashboard, /seller, /checkout
- Structured data: Product schema on product pages, BreadcrumbList on catalog pages, Review schema on reviews
- Page load speed: LCP < 2.5s, FID < 100ms, CLS < 0.1 (Core Web Vitals)
- Image optimization: Next.js Image component with WebP/AVIF, lazy loading, responsive sizes
- HTTPS enforced with HSTS header (max-age=31536000, includeSubDomains, preload)
- URLs are lowercase, hyphenated, no trailing slash (or always trailing slash — pick one)

SECTION 10

Security Checklist

Security is designed in from day 1, mapped against the OWASP Top 10 (2021). This section covers the ten standard OWASP risks plus three project-specific security concerns: APK download URL signing, payment webhook verification, and PCI-DSS scope minimization.

10.1 OWASP Top 10 Controls

OWASP Risk	Threat	NestJS/Next.js Control
A01: Broken Access Control	User accesses another user's order; reviewer approves product outside their subcategory	Casl.js RBAC with resource ownership checks. Reviewer guard validates subcategory assignment. Every controller method annotated with @Roles + resource ownership filter.
A02: Cryptographic Failures	Password hash leak, payment credentials in plaintext, JWT in localStorage	bcrypt cost 12. Payment credentials in AWS Secrets Manager. JWT in httpOnly+Secure+SameSite cookies. RDS encrypted at rest. S3 bucket policies deny non-TLS.
A03: Injection	SQL injection via search, NoSQL injection via filter params	Prisma parameterized queries (no string concatenation). Zod validation on every input. Meilisearch queries use escaped search syntax.
A04: Insecure Design	Missing rate limits, no account lockout, no audit log	Rate limit: 5 login attempts/min/IP, 100 API req/min/user. Account lockout after 10 failed attempts. Audit log every admin action, payment action, refund action.
A05: Security Misconfiguration	Default headers missing, CORS too permissive, error messages leak stack traces	Helmet middleware (CSP, HSTS, X-Frame-Options, X-Content-Type-Options). CORS allowlist specific origins. Global exception filter returns generic error in production, full stack in staging.
A06: Vulnerable Components	npm packages with known CVEs	npm audit in CI (blocks deploy on high/critical). Dependabot enabled. Renovate bot for major updates. Snyk for continuous monitoring.
A07: Auth Failures	Credential stuffing, session fixation, JWT theft	Rate-limited login. bcrypt cost 12. JWT rotation on sensitive actions. Refresh token rotation. Optional 2FA (TOTP) for admins and reviewers.
A08: Data Integrity Failures	Unsigned JWT, untrusted deserialization	JWT signed with RS256 (asymmetric). All inputs validated with Zod schemas. No eval() or Function(). Webhook payloads signature-verified.
A09: Logging Failures	No audit trail, logs not searchable, no alerts on anomalies	Structured JSON logs with correlation IDs. CloudWatch Logs Insights for search. Alarms on error rate > 1%, login failures > 50/min, payment failures > 10/min.
A10: SSRF	Server fetches user-provided URLs (e.g., preview_video, demo_url)	URL allowlist (youtube.com,youtu.be for preview_video). DNS rebinding protection. Outbound requests restricted to allowlist via VPC egress controls.

APK Download URL Signing

The site serves APK files for download. Currently URLs are direct file paths (e.g., /storage/products/123/game.apk). Migration target: AWS S3 presigned URLs with 1-hour expiry. Each download link is generated on-demand, includes signature + expiration, and is bound to the logged-in user's purchase record. No anonymous APK downloads possible. Download attempts logged with user_id, product_id, ip_address, user_agent, timestamp.

Payment Webhook Security

Razorpay webhooks: verify X-Razorpay-Signature HMAC SHA256 against webhook secret. PayPal IPN: verify with PayPal's verify API call (no signature). Manual UPI: no webhook (admin manually marks as paid). All webhook handlers are idempotent — re-processing the same webhook event has no side effects. Webhook payloads stored in payment_events table for audit. Webhook endpoints accept POST only, no auth required (signature is the auth), rate-limited to gateway IP ranges only.

Payment Card Data

The platform never touches raw card data. Razorpay and Stripe Checkout collect card data on their own domains (iframe/redirect). Our server only receives a token/payment_id. PCI-DSS SAQ-A scope. No card data stored, logged, or transmitted through our infrastructure.

SECTION 11

Risk Register & Open Questions

Twelve risks have been identified and ranked by likelihood and impact. The most critical risk (R1) is the exposure of payment gateway credentials in the SQL dump — this is a P0 security incident that must be remediated before any code is written. Twenty open questions remain blocked on receipt of the Laravel source code; these will be answered in Phase 0 Part B.

11.1 Risk Register

#	Risk	Likelihood	Impact	Mitigation
R 1	Exposed payment gateway keys in DB dump (Razorpay live, PayPal secret, Google OAuth)	CRITICAL	CRITICAL	Rotate ALL exposed keys BEFORE writing any code. Rotate: Razorpay key_id+key_secret, PayPal client_id+client_secret, Google OAuth client_secret. Treat the SQL dump as compromised.
R 2	Laravel source code not provided — admin flows and validation rules unknown	HIGH	HIGH	Block Phase 5 (Admin) kickoff until source code is provided. Use SQL schema + live admin screenshots as interim reference. Flag any assumptions in Migration Decision Records.
R 3	SEO ranking drops during cutover	MEDIUM	HIGH	100% URL preservation. 301 redirects for any changed URLs. Sitemap submitted 24h before cutover. GSC monitored daily for 14 days post-cutover. Rolling rollback if organic traffic drops > 20%.
R 4	Password hash migration fails for edge cases (legacy hashes, corrupted rows)	LOW	MEDIUM	Pre-migration audit: scan all password hashes for non-bcrypt format. Quarantine invalid hashes. Force password reset for affected users with email notice. Less than 0.1% expected.
R 5	File storage migration (APK files + images) — large total size unknown	MEDIUM	MEDIUM	Inventory total file size before Phase 3. Use S3 batch operations for migration. Preserve original filenames + paths to avoid URL changes. CloudFront cache invalidated per-file.
R 6	Payment webhook failure post-cutover (signature verification, IP changes)	MEDIUM	HIGH	Test webhooks in sandbox 2 weeks before cutover. Verify gateway IP allowlists. Have manual payment verification runbook. Keep Laravel webhook endpoint active 7 days post-cutover as fallback.
R 7	Meilisearch scaling bottleneck at 10k+ concurrent searchers	LOW	MEDIUM	Meilisearch handles 10k+ QPS on c6i.2xlarge for our index size (~1M documents). Monitor p99 latency. Auto-scale Meilisearch vertically if needed. Fallback: route to read replica.
R 8	Image optimization regression — Next.js Image component incompatible with existing URLs	MEDIUM	LOW	Use Next.js Image with `unoptimized` fallback for legacy URLs. Migrate images to S3 + CloudFront with on-the-fly resize via Lambda@Edge. Test on staging with production image set.
R 9	Multi-currency rounding errors (USD vs INR conversion at rate=88)	MEDIUM	MEDIUM	Use Decimal.js (not JS Number) for all money math. Store all amounts in lowest unit (cents/paise). Convert at display time. Audit 100 sample orders for rounding parity old vs new.
R 10	Support ticket attachment migration fails (broken links, missing files)	LOW	LOW	Inventory support_attachments table. Copy files to S3 preserving path structure. Verify every attachment URL resolves post-migration. Quarantine broken attachments.

#	Risk	Likelihood	Impact	Mitigation
R 1 1	Refund approval workflow regression causes buyer complaints	MEDIUM	HIGH	Refund flow tested end-to-end on staging with 20 sample refunds. Admin trained on new UI 1 week before cutover. Old Laravel admin kept warm for 7 days post-cutover for emergency refunds.
R 1 2	Cron jobs migration breaks scheduled tasks (email sends, expirations)	MEDIUM	MEDIUM	Inventory all cron_jobs from database. Map each to AWS EventBridge + Lambda or BullMQ repeatable. Run both systems in parallel for 1 week. Compare execution logs. Cutover only when parity confirmed.

11.2 Open Questions (Blocked on Laravel Source)

The following twenty questions cannot be definitively answered from the SQL schema alone. Each will be resolved in Phase 0 Part B once the Laravel source code (controllers, FormRequest validators, Blade templates, Jobs, Console commands) is shared.

1. Exact Laravel middleware stack (auth, guest, admin, reviewer, throttle, etc.)
2. Validation rules per controller (FormRequest classes)
3. Blade templates for admin panel (admin layout, dashboard, product approval, user management)
4. Email template rendering logic (how shortcodes like {{fullname}} are substituted)
5. Queued jobs (Laravel Jobs — what's queued, what's sync, what's the queue connection)
6. Scheduled tasks (cron_jobs table references — what does each cron actually do)
7. Laravel Scout / search configuration (if any — confirms Meilisearch vs alternatives)
8. Socialite provider details (Google OAuth scopes, redirect URLs, additional providers)
9. File upload rules (max size, allowed MIME types, virus scanning, image processing)
10. Coupon calculation logic (stacking, expiry, usage limits — edge cases)
11. Seller fee calculation (how seller_fee vs buyer_fee is computed per category)
12. Refund approval workflow (who approves, what triggers seller notification, money flow)
13. Reviewer subcategory assignment logic (how reviewers.subcategories is enforced)
14. Author level promotion logic (when does a user level up — based on what event)
15. Notification template rendering (which template for which event, multi-language support)
16. Existing API routes (routes/api.php — what endpoints already exist for the Next.js is_web flag)
17. Admin role permissions (Casl abilities, what each admin role can/cannot do)
18. Blog post body rendering (Markdown, HTML, WYSIWYG — affects XSS protection strategy)
19. Product slug generation (auto from title, manual, unique constraint enforcement)

20. Cart session ID generation (how session_id is created for guest carts)

SECTION 12

Next Steps & Sign-off

Immediate Next Actions (within 7 days)

Three actions are required to unlock Phase 0 Part B and Phase 1 kickoff. First, rotate all exposed payment gateway credentials — Razorpay live `key_id` and `key_secret`, PayPal `client_id` and `client_secret`, Google OAuth `client_secret`. Treat the SQL dump as a compromised credential set. Second, share the Laravel source code (`composer.json`, `app/`, `routes/`, `database/migrations`, `config/`, `resources/views/`) plus `.env.example`. Third, provide admin panel access (staging credentials OR screen recordings of every admin page) so admin flows can be documented.

Phase 0 Part B Scope (once source code is received)

Part B will cover: (1) Controller-by-controller business logic documentation, (2) Validation rules extracted from `FormRequest` classes, (3) Admin panel flow diagrams, (4) Email template rendering logic, (5) Queued jobs inventory, (6) Scheduled tasks (cron) inventory, (7) API route inventory (existing `is_web` endpoints), (8) Middleware stack documentation, (9) Updated Schema Migration Decision Record with source-code-informed refinements, (10) Updated Risk Register. Estimated time to produce Part B: 3-4 hours after source code receipt.

Phase 1 Kickoff Prerequisites

Phase 1 (Foundation) can kick off once Part A is approved AND Part B is delivered. Prerequisites: (1) AWS account provisioned with appropriate IAM roles. (2) GitHub repo created with branch protection rules. (3) Vercel team account connected to GitHub. (4) Domain DNS access (Route 53 or current registrar). (5) Razorpay and PayPal sandbox accounts for testing. (6) Sentry project created. (7) Engineering team briefed on architecture decisions in this document.

Communication Protocol

Weekly status meeting (30 min) every Monday. Decision log maintained in GitHub Discussions — every architecture decision recorded with rationale and date. Escalation path: engineer -> tech lead -> principal architect -> product owner. Daily standup in Slack `#readygamecode-migration`. Blockers flagged within 4 hours. Phase exit criteria reviewed at end of each phase before proceeding to next.

Sign-off Block

This Phase 0 Discovery Document (Part A) requires sign-off from: (1) Principal Architect (author), (2) Product Owner / Business Stakeholder, (3) Engineering Lead. Sign-off indicates agreement with the locked decisions, architecture recommendations, migration roadmap, and risk register. Any objection must be raised in writing within 5 business days of receipt. After sign-off, Part B can commence upon source code receipt, and Phase 1 kickoff can be scheduled.

Sign-off

Role	Name	Signature	Date
Principal Architect (Author)			
Product Owner / Business Stakeholder			
Engineering Lead			

SECTION A

Appendix A: Complete 67-Table Inventory

All 67 tables in the existing MySQL database, grouped by domain and mapped to the target NestJS module. Decision values: KEEP / KEEP + ADD COLUMN / REFACTOR / RENAME / EVALUATE / DEPRECATE. Tables marked EVALUATE require source code review to confirm purpose.

#	Table	Domain	Target Module	Decision
1	activities	Admin/Workflow	Activity	KEEP
2	admins	Auth/Users	Admin (Auth)	KEEP
3	admin_notifications	Notifications	Notification	KEEP
4	admin_password_resets	Auth/Users	Admin (Auth)	KEEP (or merge with password_resets)
5	advertisements	Content	Page	KEEP
6	api_keys	Settings/System	Settings	KEEP
7	author_levels	Admin/Workflow	Author	KEEP
8	author_level_user	Auth/Users	Author	KEEP (pivot)
9	blog_categories	Content	Blog	KEEP
10	blog_posts	Content	Blog	KEEP
11	campaigns	Commerce	Marketing (new)	KEEP
12	campaign_products	Commerce	Marketing (new)	KEEP (pivot)
13	carts	Commerce	Cart	REFACTOR (introduce Cart parent)
14	categories	Catalog	Category	KEEP
15	changelogs	Catalog	Product	KEEP
16	collection_product	Catalog	Product	KEEP (pivot)
17	comments	Reviews/Social	Review	KEEP
18	coupons	Commerce	Order	KEEP
19	cron_jobs	Settings/System	System (cron)	KEEP (or migrate to EventBridge)

#	Table	Domain	Target Module	Decision
20	cron_job_logs	Settings/System	System (cron)	KEEP
21	cron_schedules	Settings/System	System (cron)	KEEP
22	deposits	Commerce	Payment	RENAME (deposits -> payments)
23	device_tokens	Auth/Users	User	KEEP (for push notifications)
24	downloads	Commerce	Media	KEEP
25	extensions	Catalog	Product	KEEP
26	followers	Reviews/Social	Author	KEEP
27	forms	Catalog	DynamicForm	KEEP
28	frontends	Content	Page	KEEP
29	gateways	Payments	Payment	KEEP
30	gateway_currencies	Payments	Payment	KEEP
31	general_settings	Settings/System	Settings	KEEP (singleton)
32	languages	Settings/System	Settings	KEEP
33	migrations	Settings/System	—	DEPRECATE (Prisma handles)
34	notification_logs	Notifications	Notification	RENAME -> NotificationLog
35	notification_templates	Notifications	Notification	KEEP
36	orders	Commerce	Order	KEEP + ADD COLUMNS
37	order_items	Commerce	Order	KEEP
38	pages	Content	Page	KEEP
39	password_resets	Auth/Users	User (Auth)	KEEP

#	Table	Domain	Target Module	Decision
40	products	Catalog	Product	KEEP + ADD COLUMNS
41	product_collections	Catalog	Product	KEEP
42	product_rating	Catalog	Review	KEEP (or merge with reviews)
43	product_user	Catalog	Order (UserPurchase)	KEEP
44	product_views	Catalog	Product	EVALUATE (maybe Redis)
45	ratings	Reviews/Social	Review	KEEP (or merge with reviews)
46	refund_activities	Commerce	Order (Refund)	KEEP
47	refund_requests	Commerce	Order (Refund)	KEEP
48	rejections	Admin/Workflow	Activity	KEEP
49	reported_reviews	Reviews/Social	Review	KEEP
50	reported_reviews_attachments	Reviews/Social	Review	KEEP
51	reviewers	Auth/Users	Reviewer (Auth)	KEEP
52	reviewer_password_resets	Auth/Users	Reviewer (Auth)	KEEP
53	reviewer_sub_category	Auth/Users	Reviewer (Auth)	KEEP (pivot)
54	reviews	Reviews/Social	Review	KEEP
55	review_categories	Reviews/Social	Review	KEEP
56	subscribers	Content	Subscriber	KEEP
57	sub_categories	Catalog	Category	KEEP
58	support_attachments	Support/Moderation	Support	KEEP
59	support_messages	Support/Moderation	Support	KEEP

#	Table	Domain	Target Module	Decision
60	support_tickets	Support/Moderation	Support	KEEP
61	transactions	Commerce	Order (Ledger)	KEEP
62	update_logs	Settings/System	System (updates)	DEPRECATE (Laravel-specific)
63	users	Auth/Users	User (Auth)	KEEP + ADD COLUMNS
64	user_follows	Reviews/Social	Author	EVALUATE (duplicate of followers?)
65	user_logins	Auth/Users	User	KEEP (login audit)
66	withdrawals	Commerce	Order (Withdrawal)	KEEP
67	withdraw_methods	Commerce	Order (Withdrawal)	KEEP

SECTION B

Appendix B: Active Payment Gateway Configuration

All payment gateway entries in the gateways table. Only 3 are currently active (status=1): Razorpay for INR, PayPal for USD, and Google Pay for manual UPI. The remaining 16 are disabled but schema-preserved for future activation. Sensitive credentials (key_secret, client_secret) are redacted in this document.

P0 SECURITY INCIDENT: The original SQL dump contains live production credentials for Razorpay (rzp_live_*), PayPal (live client_id + client_secret), and Google OAuth (live client_secret). All three MUST be rotated immediately before any further action. Treat the SQL dump file as compromised and delete all copies once rotation is complete.

ID	Gateway	Code	Currency	Status	Notes
1	Paypal (direct)	101	USD + 24 currencies	ACTIVE	Email: paras.bhalodiya.developer@gmail.com. IPN-based. Deprecated by PayPal — recommend migrate to Stripe Checkout.
10	RazorPay	110	INR only	ACTIVE	LIVE keys (rzp_live_*) — ROTATE IMMEDIATELY. Webhook signature: X-Razorpay-Signature HMAC SHA256.
63	Google Pay (manual)	1000	INR (manual)	ACTIVE	QR code (tagadiyainfotech.jpg) shown to buyer. Buyer sends screenshot. Admin manually marks as paid. form_id=4.
3	Stripe Hosted	102	USD + 18 currencies	DISABLED	TEST keys (sk_test_*). Recommend enable as Stripe Checkout post-cutover to replace PayPal direct.
11	Stripe Storefront (StripeJs)	111	USD + 18 currencies	DISABLED	TEST keys. Alternative Stripe integration. Recommend enable.
25	Stripe V3 Checkout	114	USD + 18 currencies	DISABLED	TEST keys + webhook endpoint secret. Best Stripe integration. RECOMMENDED for activation.
24	Paypal Express (PaypalSdk)	113	USD + 24 currencies	DISABLED	PayPal SDK integration. Alternative to gateway ID 1.
64	Paypal (new)	—	USD	ACTIVE	Newer PayPal config (added Oct 2025). Same email as ID 1. Likely replacement for ID 1.

ID	Gateway	Code	Currency	Status	Notes
2	Perfect Money	—	USD, EUR	DISABLE D	Not used in production
4	Skrill	—	40 currencies	DISABLE D	Not used in production
5	PayTM	—	52 currencies	DISABLE D	Not used in production
6	Payeer	—	USD, EUR, RUB	DISABLE D	Not used in production
7	PayStack	—	USD, NGN	DISABLE D	Not used in production. Good for Africa expansion
9	Flutterwave	—	24 currencies	DISABLE D	Not used. Good for Africa expansion
12	Instamojo	—	INR	DISABLE D	Not used in production
13	Blockchain	—	BTC	DISABLE D	Crypto. Not used
15	CoinPayments	—	100+ cryptos	DISABLE D	Crypto. Not used
16	CoinPayments Fiat	—	23 fiat	DISABLE D	Not used
17	Coingate	—	USD, EUR	DISABLE D	Crypto. Not used
18	Coinbase Commerce	—	All currencies	DISABLE D	Crypto. Not used
27	Mollie	—	30 currencies	DISABLE D	Not used. Good for EU expansion
30	Cashmaal	—	PKR, USD	DISABLE D	Not used
36	Mercado Pago	—	11 currencies	DISABLE D	Not used. Good for LATAM expansion
37	Authorize.net	—	11 currencies	DISABLE D	Not used
46	NMI	—	35 currencies	DISABLE D	Not used
50	BTCPay	—	BTC, LTC	DISABLE D	Self-hosted crypto. Not used
51	NowPayments Hosted	—	50+ cryptos	DISABLE D	Crypto. Not used
52	NowPayments Checkout	—	USD, EUR	DISABLE D	Crypto. Not used

ID	Gateway	Code	Currency	Status	Notes
53	2Checkout	—	100+ currencies	DISABLED	Not used
54	Checkout (Checkout.com)	—	10 currencies	DISABLED	Not used
59	Binance	—	BTC, USD, BNB	DISABLED	Crypto. Not used
60	SslCommerz	—	BDT + 5	DISABLED	Not used. Good for Bangladesh expansion
61	Aamarpay	—	BDT	DISABLED	Not used. Bangladesh
62	bKash	—	BDT	DISABLED	Not used. Bangladesh

SECTION C

Appendix C: Glossary

Key terms used throughout this document. Definitions are intentionally brief and tailored to the Ready Game Code migration context.

Term	Definition
Strangler-Fig	Gradual migration pattern where new system grows around the old, eventually replacing it. Coined by Martin Fowler. Each feature is migrated independently, allowing rollback per-route.
RBAC	Role-Based Access Control. Authorization model where permissions are assigned to roles, and users are assigned roles. Implemented in NestJS via Casl.js.
DDD	Domain-Driven Design. Software design approach that focuses on modeling the business domain. Bounded contexts, entities, value objects, aggregates.
Clean Architecture	Architecture where business logic is independent of frameworks, UI, and databases. Layers: entities, use cases, interface adapters, frameworks & drivers.
Prisma	TypeScript ORM with schema-first DSL, type-safe query builder, and migration system. Alternative to TypeORM. Chosen for this migration.
NestJS Module	Organization unit in NestJS. Groups related controllers, services, providers. Each module is a bounded context.
JWT	JSON Web Token. Compact, URL-safe token format for stateless auth. Three parts: header, payload, signature.
httpOnly Cookie	Cookie attribute that prevents JavaScript access. Mitigates XSS-based token theft. Used for JWT storage in this architecture.
Meilisearch	Open-source search engine written in Rust. Sub-50ms search, typo tolerance, easy self-host. Alternative to Elasticsearch, Algolia.
ECS Fargate	AWS container runtime that doesn't require EC2 management. Pay-per-container. Auto-scales. Best for long-running NestJS APIs.
RDS Multi-AZ	AWS RDS deployment with synchronous standby in another Availability Zone. Automatic failover. ~60 seconds downtime during failover.
CloudFront	AWS CDN. Caches content at 410+ edge locations. Used for static assets, image delivery, and DDoS protection.
SQS	Amazon Simple Queue Service. Managed message queue. Used for decoupling async jobs (email sending, image processing) from API requests.

Term	Definition
SES	Amazon Simple Email Service. Transactional email service. 62,000 free emails/month if sent from EC2. Replaces PHP mail().
OWASP Top 10	Open Web Application Security Project's list of top 10 web application security risks. Updated every 3-4 years. Industry standard for security audits.
KYC	Know Your Customer. Identity verification process for regulatory compliance. Present in users table (kv status, kyc_data) but disabled in production.
2FA	Two-Factor Authentication. Requires second verification beyond password. TOTP (Time-based One-Time Password) is most common. Present in users table (ts, tv fields) but disabled.
Bcrypt Cost Factor	Work factor for bcrypt hashing. Higher = slower = more secure. Cost 12 = ~250ms per hash. Production recommendation: 12-14.
OAuth 2.0	Authorization framework allowing third-party apps to access user data without sharing passwords. Used for Google login.
Socialite	Laravel's OAuth authentication package. Supports Google, Facebook, Twitter, GitHub, etc. NestJS equivalent: Passport.js with strategy packages.
Paged.js	JavaScript library that polyfills CSS Paged Media spec in browsers. Used by html2pdf-next.js for HTML-to-PDF pagination.
ReportLab	Python library for programmatic PDF generation. Used by the pdf skill's report brief for structured documents like this one.
Idempotency Key	Client-generated unique key sent with API requests to prevent duplicate processing. Critical for payment endpoints — same key + same payload = same result, no double-charge.
Webhook	HTTP callback. Third-party service (Razorpay, PayPal) sends POST request to your endpoint when an event occurs (payment captured, refund processed). Must be idempotent and signature-verified.
Casl.js	TypeScript authorization library. Defines abilities per role (e.g., admin can update any product, user can update only their own products). Integrated with NestJS via decorators.